Task 1: Bash Scripting – Assignment 01

Date: 20261023

Outcome

Successfully practiced Bash scripting on an Amazon Linux 2023 EC2 instance. Created and executed scripts for basic commands, system information, variables, user input, and file renaming. Also practiced shell identification, file permissions, and troubleshooting common Bash scripting errors.

Objective

To learn Bash scripting fundamentals and understand how to automate Linux administration tasks using shell commands, variables, user input, and file operations. These skills are useful for automating repetitive activities in DevOps workflows.

Prerequisites

- AWS account and an active EC2 instance.
- Amazon Linux 2023 operating system.
- SSH access using MobaXterm and a private key.
- Basic Linux command-line knowledge.
- Vim text editor.
- Bash shell.

Environment Details

Parameter	**Value**
Cloud platform	AWS EC2
Operating system	Amazon Linux 2023
Hostname	ip-172-31-31-187
User	Root
Working directory	/bash/scripts
Shell	Bash
Editor	Vim
Assignment	Bash-Scripting Assignment 01

Step-by-Step Implementation

Step 1: Access the EC2 Instance and Switch to Root

Connected to the EC2 instance through SSH using MobaXterm.

Command:

sudo su

Verified access to the root shell.

Explanation: The sudo su command switches the current session to the root user, allowing administrative operations.

Step 2: Create a Test File and Execute System Commands

Created a file named test using Vim:

vim test

Checked the file details:

ll

Changed permissions:

chmod 777 test

Executed the file:

./test

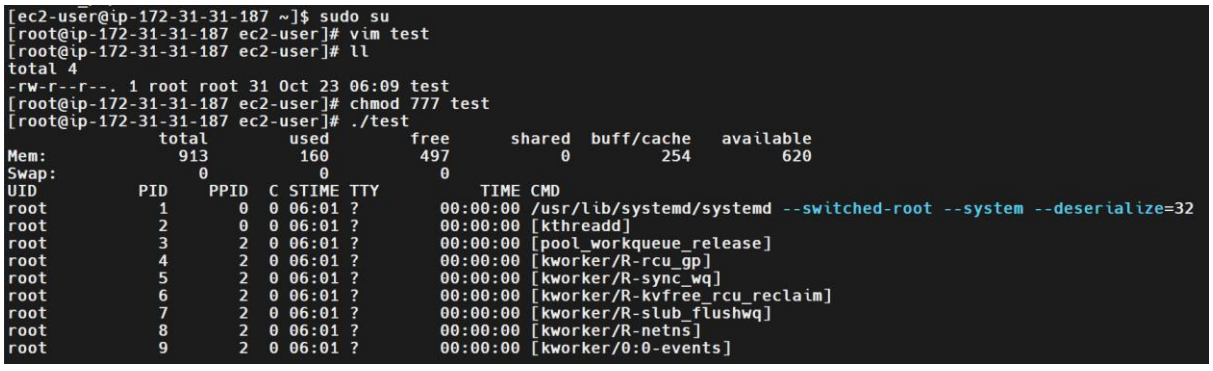
The output displayed system information, including memory usage, running processes, and filesystem details.

Explanation:

- vim test creates or opens a file for editing.
- ll displays file permissions, ownership, size, and modification time.
- chmod 777 test grants read, write, and execute permissions to everyone.
- ./test executes the file from the current directory.

Note: Avoid 777 permissions in production. Use only the permissions required by the script.

Screenshot Evidence



Step 3: Identify the Current Shell

Executed:

echo $0

Output:

bash

Explanation: The $0 variable displays the name used to invoke the current shell or script. In this example, the shell is Bash.

Step 4: List the Available Shells

Executed:

cat /etc/shells

The output included:

/bin/sh

/bin/bash

/usr/bin/sh

/usr/bin/bash

/bin/csh

/bin/tcsh

/usr/bin/csh

/usr/bin/tcsh

Explanation: The /etc/shells file lists shells recognized by the system as valid login shells.

Step 5: Test Different Shells

Tested the following shells individually:

sh

echo $0

exit

csh

echo $0

tcsh

echo $0

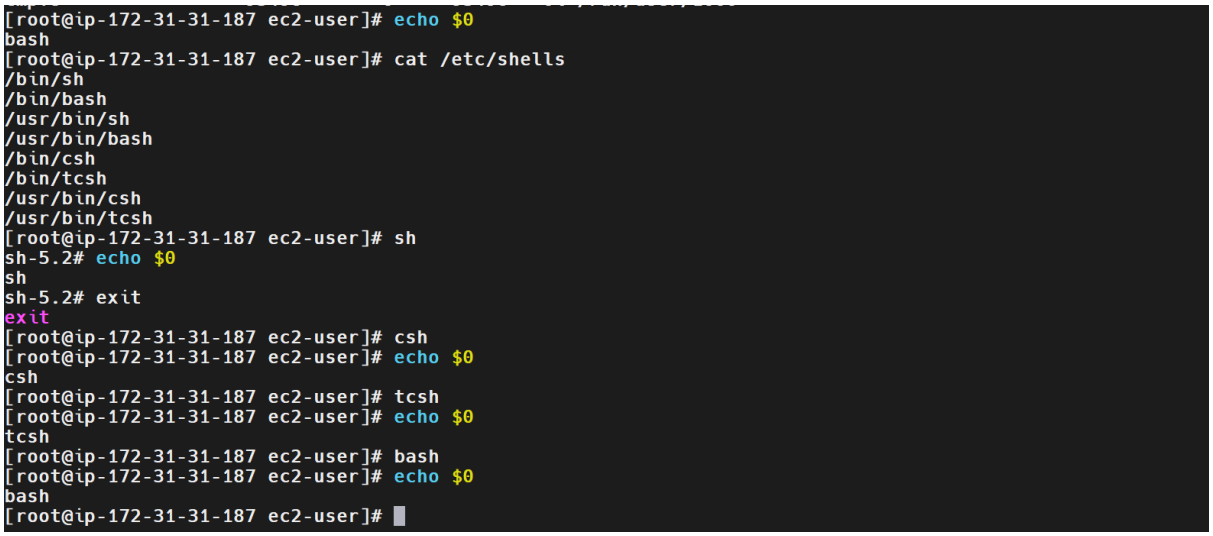
bash

echo $0

The outputs identified the respective shells as sh, csh, tcsh, and bash.

Explanation: A shell interprets commands entered by the user. Different shells provide different features and syntax. The exit command exits the current shell session and returns to the previous shell.

Screenshot Evidence



Step 6: Create the Bash Scripts Directory

Checked the current location:

pwd

Output:

/home/ec2-user

Created the directory:

mkdir -p /bash/scripts

An attempt to use the relative path failed:

cd bash/scripts

Error:

bash: cd: bash/scripts: No such file or directory

Corrected the path by using:

cd /bash/scripts

Verified the location:

pwd

Output:

/bash/scripts

Explanation: An absolute path starts from /, while a relative path is interpreted from the current working directory. Using /bash/scripts avoids ambiguity about the current location.

Screenshot Evidence

-

```
[root@ip-172-31-31-187 ec2-user]# pwd
/home/ec2-user
[root@ip-172-31-31-187 ec2-user]# mkdir -p /bash/scripts
[root@ip-172-31-31-187 ec2-user]# cd bash/scripts
bash: cd: bash/scripts: No such file or directory
[root@ip-172-31-31-187 ec2-user]# cd /bash/scripts
[root@ip-172-31-31-187 scripts]# ls
[root@ip-172-31-31-187 scripts]# ls -l
total 0
[root@ip-172-31-31-187 scripts]# pwd
/bash/scripts
[root@ip-172-31-31-187 scripts]# vim helloworld
[root@ip-172-31-31-187 scripts]# ll
total 4
-rw-r--r--. 1 root root 34 Oct 23 06:28 helloworld
[root@ip-172-31-31-187 scripts]# chmod 777 helloworld
[root@ip-172-31-31-187 scripts]# ./helloworld
Hello world
[root@ip-172-31-31-187 scripts]# cd
[root@ip-172-31-31-187 ~]# pwd
/root
[root@ip-172-31-31-187 ~]# cd /bash/scripts/helloworld
bash: cd: /bash/scripts/helloworld: Not a directory
[root@ip-172-31-31-187 ~]#  /bash/scripts/helloworld
Hello world
[root@ip-172-31-31-187 ~]# cd /bash/scripts
[root@ip-172-31-31-187 scripts]# ll
total 4
-rwxrwxrwx. 1 root root 34 Oct 23 06:28 helloworld
[root@ip-172-31-31-187 scripts]#
```

Step 7: Create and Execute a Hello World Script

Created a script:

vim helloworld

Added the following content:

#!/bin/bash

echo "Hello world"

Changed its permissions:

chmod 777 helloworld

Executed it:

./helloworld

Output:

Hello world

Renamed the script:

mv helloworld helloworld.bash

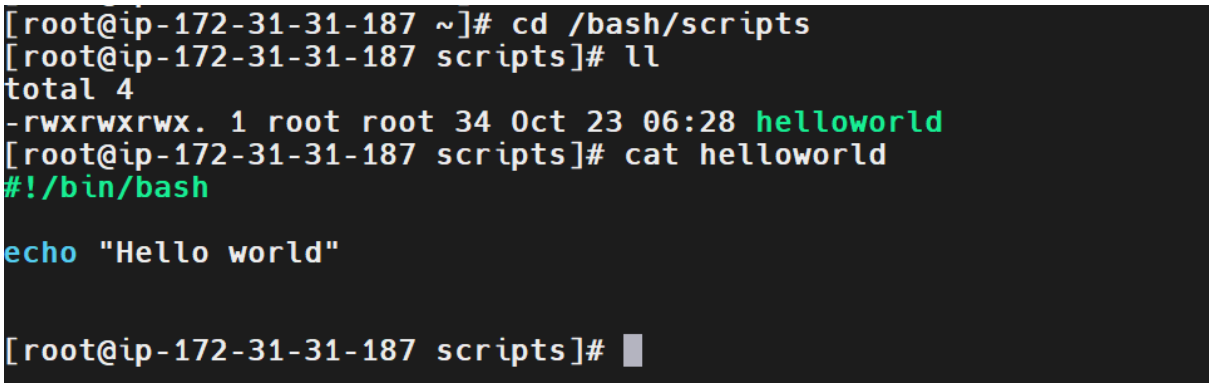
Verified the file:

ll

Explanation:

- #!/bin/bash is the shebang, specifying the Bash interpreter.
- echo displays text on the terminal.
- chmod changes file permissions.
- mv renames or moves a file.
- ./helloworld executes a script using its relative path.

Screenshot Evidence



Step 8: Understand Script Permissions and Execution

Created another script:

vim abc.bash

Script content:

#!/bin/bash

echo "abc"

Attempted direct execution:

./abc.bash

Observed error:

bash: ./abc.bash: Permission denied

Executed the script through Bash:

bash abc.bash

Output:

abc

Added execute permission:

chmod 755 abc.bash

Executed it directly:

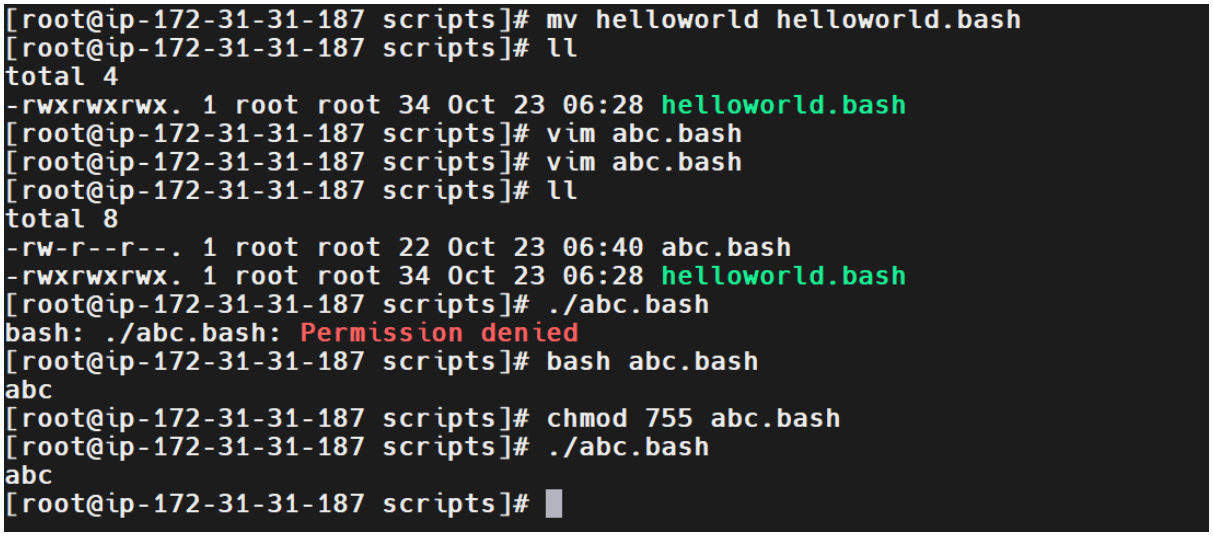
./abc.bash

Output:

abc

Explanation: Direct execution requires execute permission. Running bash abc.bash invokes the interpreter explicitly, so the script file itself does not need the execute bit. The 755 permission allows the owner to read, write, and execute the file, while others can read and execute it.

Screenshot Evidence



Step 9: Add Comments and Variables to a Script

Edited helloworld.bash:

vim helloworld.bash

Added the following content:

#!/bin/bash

#purpose: testing script format

#date: 23/10/2026

#modification: 23/10/2026

a="Hello world"

echo $a

Verified the content:

cat helloworld.bash

Explanation:

- Lines beginning with # are comments, except the shebang, which identifies the interpreter.
- a="Hello world" assigns a string to the variable a.
- echo $a displays the variable's value.

Initially, the assignment was entered with spaces around =, causing an error:

./helloworld.bash: line 6: a: command not found

Corrected it to:

a="Hello world"

Executed the script:

./helloworld.bash

Output:

Hello world

Updated the value to Hello devops and executed the script again:

Hello devops

Key learning: Bash variable assignments must not contain spaces around the equal sign.

Screenshot Evidence

```
#!/bin/bash
#purpose: testing script format
#date: 23/10/2026
#modification: 23/10/2026

a ="Hello world"

echo $a

~
~
```

```
-rwxr-xr-x. 1 root root 22 Oct 23 06:40 abc.bash
-rwxrwxrwx. 1 root root 34 Oct 23 06:28 helloworld.bash
[root@ip-172-31-31-187 scripts]# vim helloworld.bash
[root@ip-172-31-31-187 scripts]# ./helloworld.bash
./helloworld.bash: line 6: a: command not found

[root@ip-172-31-31-187 scripts]# vim helloworld.bash
[root@ip-172-31-31-187 scripts]# ./helloworld.bash
Hello world
[root@ip-172-31-31-187 scripts]# cat helloworld.bash
#!/bin/bash
#purpose: testing script format
#date: 23/10/2026
#modification: 23/10/2026

a="Hello world"

echo $a

[root@ip-172-31-31-187 scripts]# vim helloworld.bash
[root@ip-172-31-31-187 scripts]# ./helloworld.bash
Hello devops
[root@ip-172-31-31-187 scripts]#
```

Step 10: Clean the Directory and Recreate the Hello World Script

Removed the existing files from the working directory:

rm -rf *

Verified the directory:

ll

Created the script again:

vim helloworld.bash

Added the following content:

#!/bin/bash

echo "helloworld"

Changed the permissions:

chmod 755 helloworld.bash

Executed the script:

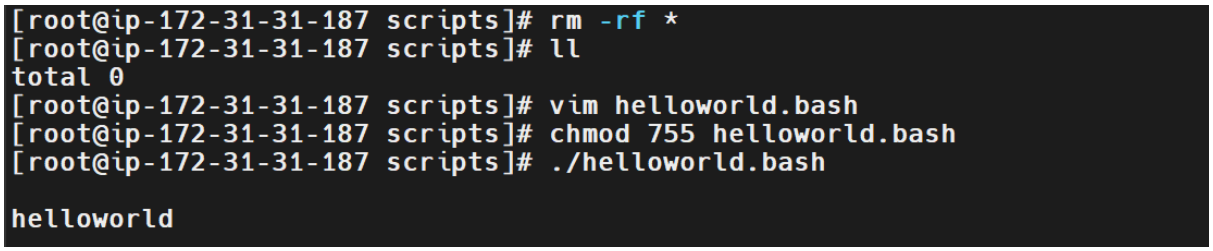
./helloworld.bash

Output:

helloworld

Explanation: rm -rf * removes matching files and directories in the current directory without prompting. This command is destructive, especially when executed as root, so always verify the current directory before running it.

Screenshot Evidence



Step 11: Create a Script to Execute Basic Linux Commands

Created:

vim commands.bash

The script executed basic commands to display the current directory, list files, identify the current user, show the date, and display a calendar.

Changed permissions and executed the script:

chmod 755 commands.bash

./commands.bash

The output included:

/bash/scripts

commands.bash helloworld.bash

root

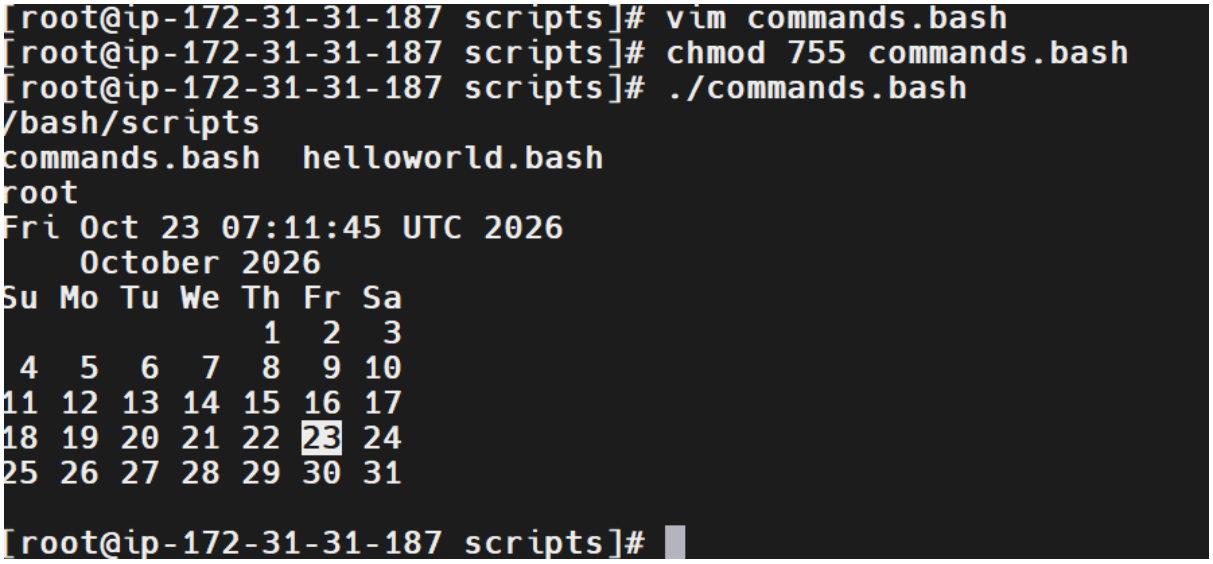
Fri Oct 23 07:11:45 UTC 2026

October 2026

The script also displayed the calendar for October 2026.

Explanation: Commands such as pwd, ls, whoami, date, and cal can be combined in a Bash script to collect system and environment information.

Screenshot Evidence



Step 12: Display System Resource Information

The system information script was executed to display information about processes, memory, CPU usage, and filesystem utilization.

The screenshot showed output from commands such as:

top

free -m

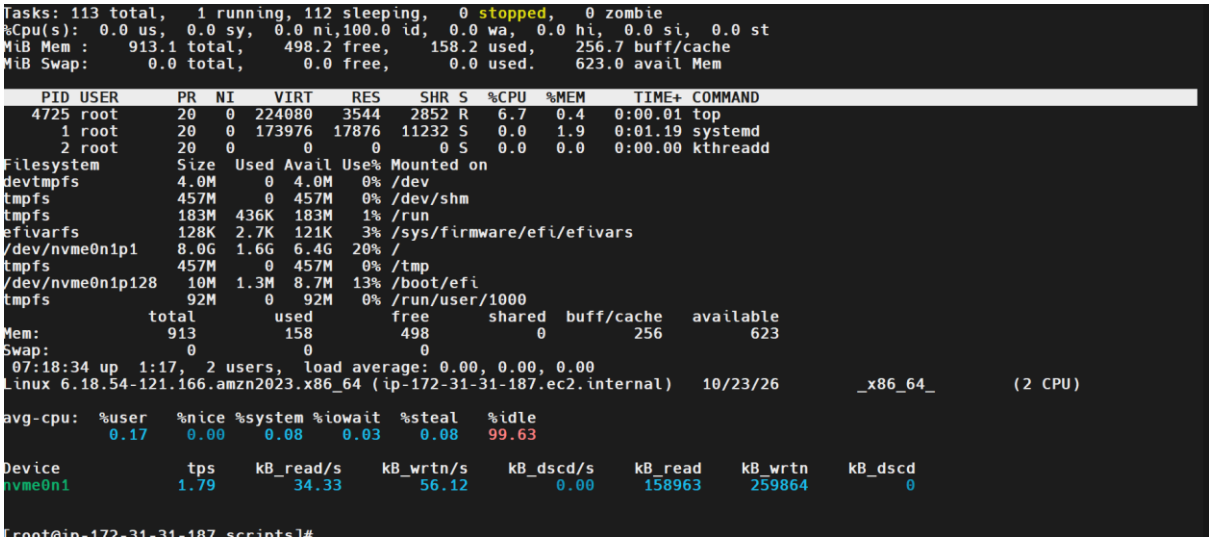
df -h

Explanation:

- top displays running processes and live resource utilization.
- free -m reports memory usage in megabytes.
- df -h displays filesystem capacity in human-readable units.

These commands are useful for troubleshooting performance issues on Linux servers.

Screenshot Evidence



Step 13: Create a Script Using Command Variables

Created:

vim variables.bash

Script content:

#!/bin/bash

#define variables

p=pwd

l=ls

wi=whoami

d=date

c='cal 2023'

echo Run variable tasks

echo

$p

$l

$wi

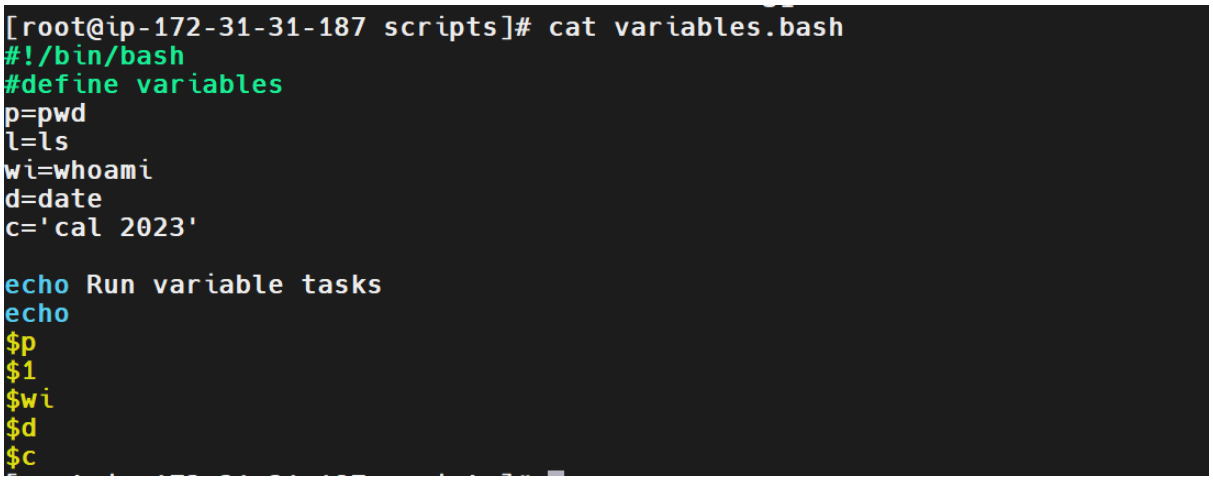
$d

$c

Explanation: This example demonstrates storing command names in variables and expanding them to execute the corresponding commands.

Important note: Storing command names in variables works for simple commands, but it is generally clearer and safer to execute commands directly or use functions. For example, use pwd instead of assigning p=pwd and then running $p.

Screenshot Evidence



Step 14: Create a Script to Accept User Input

Created:

vim input.bash

Added:

#!/bin/bash

echo 'my name is naveen'

echo

echo 'what is your name'

echo

read name

echo hello $name

Changed permissions:

chmod 755 input.bash

Executed:

./input.bash

Example output:

my name is naveen

what is your name

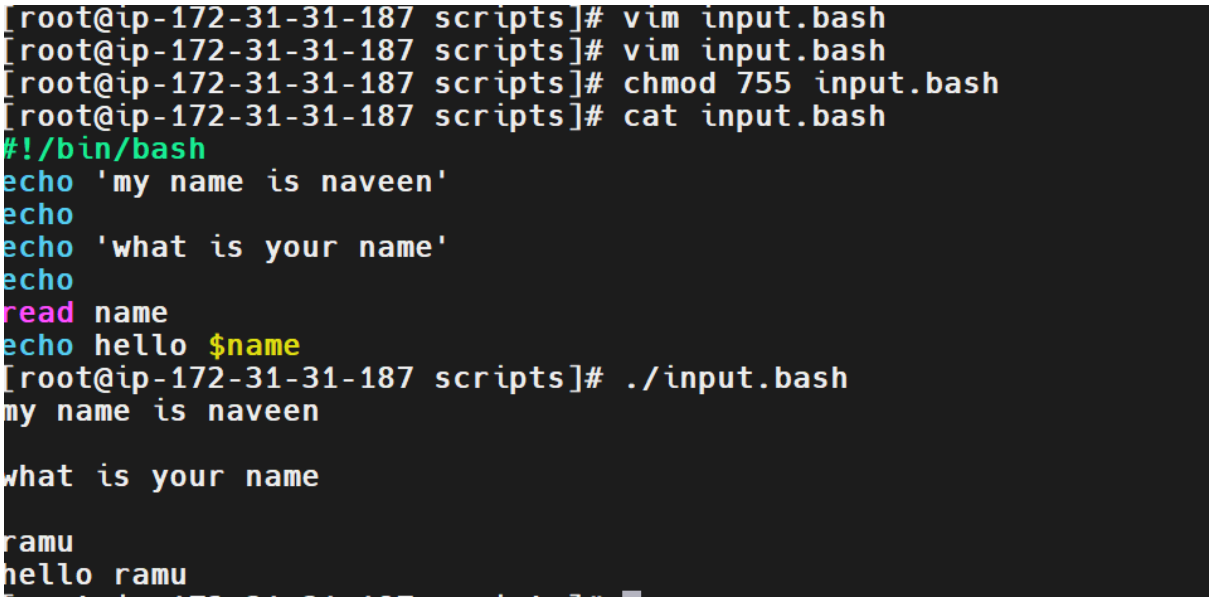
ramu

hello ramu

Explanation:

- read name accepts input from the user and stores it in the variable name.
- $name retrieves the entered value.
- The script displays a personalized greeting using the input.

Screenshot Evidence



Step 15: Create a Script Using the Hostname and User Input

Created or edited:

vim output.bash

The script was designed to display the hostname, ask for a name, and print a greeting. The hostname was checked using:

hostname

Output:

ip-172-31-31-187.ec2.internal

The script was then executed:

./output.bash

The final output included:

hello my name is ip-172-31-31-187.ec2.internal

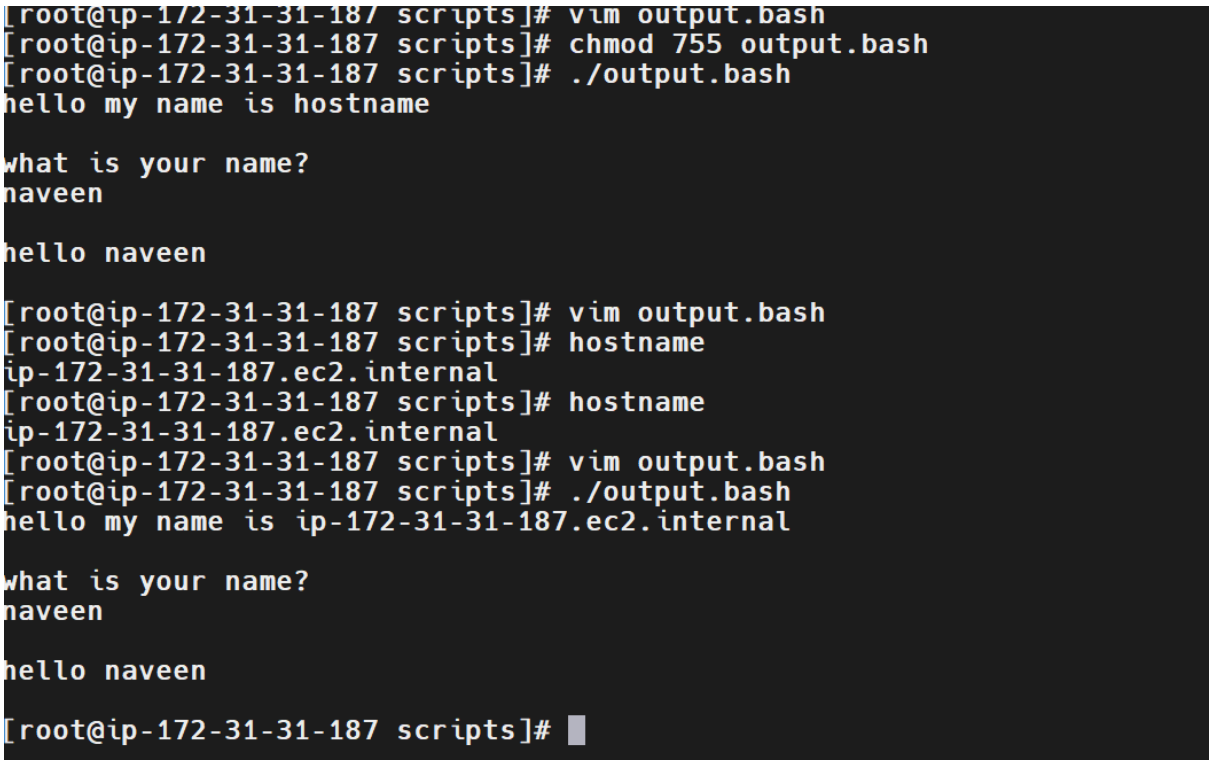
what is your name?

naveen

hello naveen

Explanation: The hostname command identifies the system's hostname. The script combines system information with interactive user input to generate output dynamically.

Screenshot Evidence



Step 16: Create a Script to Display User and System Information

Created:

vim tasks.bash

Executed:

bash tasks.bash

The output displayed information including:

root

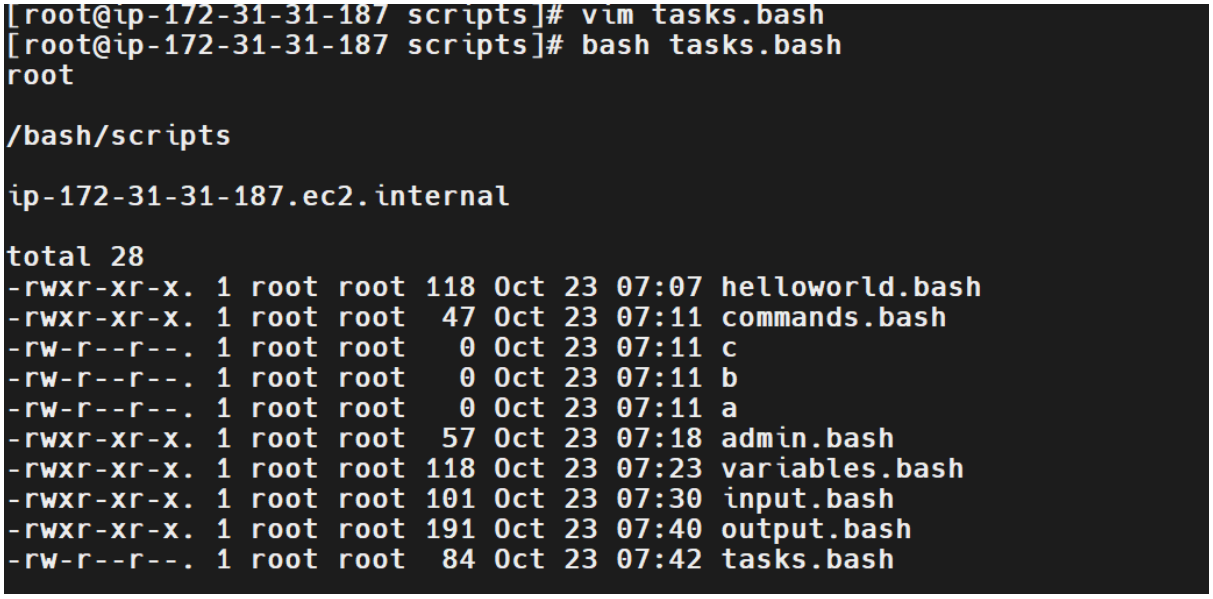
/bash/scripts

ip-172-31-31-187.ec2.internal

The output also included a listing of files in the working directory.

Explanation: This task demonstrates combining multiple Linux commands within one script to display the current user, working directory, hostname, and file details.

Screenshot Evidence



Step 17: Practice Variable Assignments

Created:

vim variables1.bash

Executed:

bash variables1.bash

Observed output:

My first name is Imran

My surname is Afzal

My surname is $c

Explanation: The script demonstrated variable usage and string output. The final line printed the literal $c, indicating that the variable was escaped or quoted in a way that prevented expansion, or that the script used a single-quoted string.

To print the value stored in c, use:

echo "My surname is $c"

Screenshot Evidence

```
[root@ip-172-31-31-187 scripts]# vim variables1.bash
[root@ip-172-31-31-187 scripts]# bash variables1.bash
My first name is Imran
My surname is Afzal
My surname is $c
[root@ip-172-31-31-187 scripts]#
```

```
[root@ip-172-31-31-187 scripts]# vim variables1.bash
[root@ip-172-31-31-187 scripts]# bash variables1.bash
My first name is Imran
My surname is Afzal
My surname is Linux class
[root@ip-172-31-31-187 scripts]#
```

Step 18: Create a Script to Collect First and Last Name

Created a file using:

vim user input

The terminal showed:

2 files to edit

This happened because the filename contained a space and Vim interpreted user and input as two separate filenames.

Executed:

bash user input

The script prompted for a first name and a last name:

What is your first name?

naveen

What is your last name?

kumar

Hello naveen kumar

Explanation: The script collected two separate inputs and combined them in a greeting.

Important lesson: When creating or accessing a filename containing spaces, enclose the name in quotes, for example:

vim "user input"

To rename it to a filename without spaces, use:

mv "user input" userinput.bash

Screenshot Evidence

```
[root@ip-172-31-31-187 scripts]# vim user input
2 files to edit
[root@ip-172-31-31-187 scripts]# bash user input
What is your first name?
naveen

What is your last name?
kumar

Hello naveen kumar
```

Step 19: Display User, Date, and Login Information

Executed a script that displayed the current user, date/time, and number of logged-in users.

Observed output:

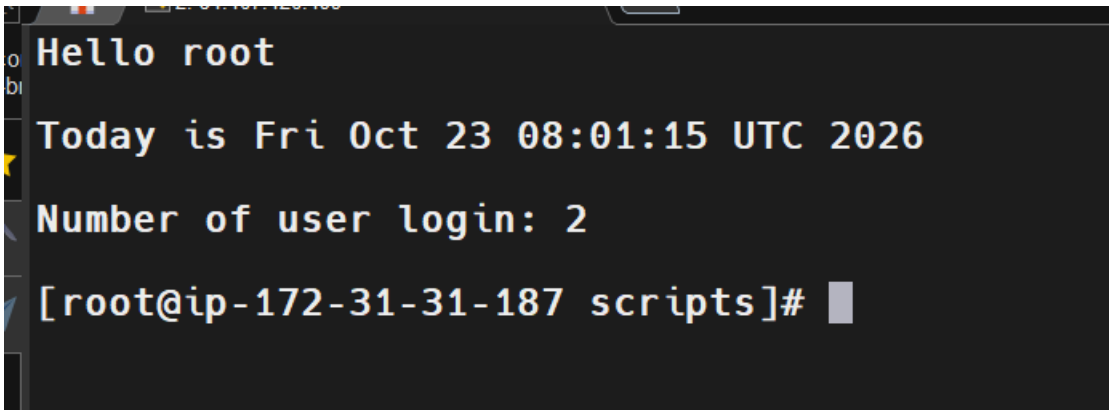
Hello root

Today is Fri Oct 23 08:01:15 UTC 2026

Number of user login: 2

Explanation: This task demonstrates collecting system information from commands and printing it in a readable format. The date and login count can be useful for basic server monitoring.

Screenshot Evidence



Step 20: Create a Script to Rename a File

Created the script:

vim ptask.bash

Executed it:

bash ptask.bash

The script prompted for the existing filename and the new filename.

Example interaction:

Enter the file name to be renamed

user

Enter the new file name

userinput.bash

The file has been renamed as userinput.bash

Explanation: The script accepts a source filename and a destination filename, then renames the file. The underlying Linux command for renaming is mv.

A basic implementation can be written as:

```
#!/bin/bash

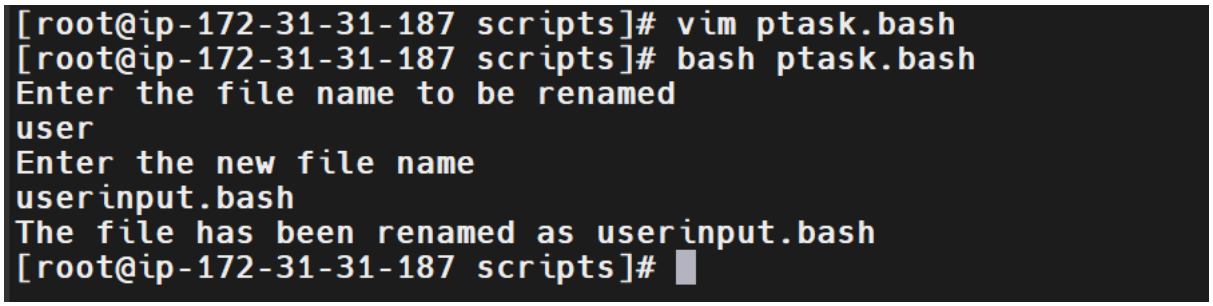
read -p "Enter the file name to be renamed: " old_name

read -p "Enter the new file name: " new_name

if [ -f "$old_name" ]; then
   mv -- "$old_name" "$new_name"
   echo "The file has been renamed as $new_name"
else
   echo "Error: File not found"
fi
```

The script checks that the original path is a regular file before attempting to rename it.

Screenshot Evidence



Validation Steps

The following results were visible in the provided screenshots.

Validation	Command or Script	Expected/Observed Result
Identify shell	echo $0	bash
List shells	cat /etc/shells	Available shell paths displayed
Verify script directory	pwd	/bash/scripts
Execute basic script	./helloworld.bash	helloworld
Test execute permissions	./abc.bash	Permission error before chmod
Execute via interpreter	bash abc.bash	abc
Execute with permissions	./abc.bash	abc
Display variable value	./helloworld.bash	Hello world / Hello devops
Display command results	./commands.bash	Directory, files, user, date, calendar
Accept user input	./input.bash	Greeting using the entered name
Display hostname	hostname	ip-172-31-31-187.ec2.internal
Collect full name	bash user input	Greeting with first and last name
Rename a file	bash ptask.bash	Confirmation of the new filename

Validation Screenshot Evidence

Attach the screenshots listed under each step. The most important evidence includes:

- Shell identification and available shells.
- Successful Hello World script execution.
- Permission-denied error and correction.
- Variable assignment and output.
- User-input and full-name scripts.
- Hostname and system information output.
- Successful file-renaming output.

Explanation

Bash scripting provides a way to automate Linux operations by combining commands, variables, user input, and file operations into reusable scripts. A script commonly begins with #!/bin/bash, which specifies the interpreter. Commands such as pwd, ls, whoami, date, hostname, and df can be used to collect useful system information.

File permissions control whether a script can be executed directly. Variables allow data to be stored and reused, while the read command makes scripts interactive. File operations such as mv allow scripts to rename files automatically. Troubleshooting issues like incorrect paths, missing execute

permissions, and spaces around variable assignments is an important part of developing reliable automation.

Real-Time Use Case

Bash scripts are commonly used by DevOps engineers to automate repetitive activities such as server health checks, log cleanup, backups, deployment preparation, and user management. For example, a script can collect hostname, memory, disk usage, and login information before a deployment. Interactive scripts can also collect filenames or environment values from an operator and perform tasks consistently.

Common Errors and Troubleshooting

Error	Cause	Resolution
No such file or directory	Incorrect relative directory path	Used the absolute path /bash/scripts
Permission denied	Execute permission missing	Used chmod 755 script_name
command not found for a variable name	Spaces around = in assignment	Used variable="value"
Vim reports 2 files to edit	Filename contains spaces but was not quoted	Used vim "user input"
Script cannot find the original file	Wrong filename or working directory	Checked pwd and verified the filename
Variable prints literally as $c	Variable expansion prevented by quoting or escaping	Used double quotes: echo "My surname is $c"
Accidental deletion of scripts	Used rm -rf *	Verify the current directory and review files before deleting

Conclusion

Successfully practiced Bash scripting fundamentals on Amazon Linux 2023, including shell identification, script creation, permissions, variables, user input, system information, and file renaming. The screenshots provide evidence of script execution and troubleshooting.